Apache Spark 4.1.1 + Kafka 4.1.1

Complete Installation, Architecture & Streaming Guide

1. Introduction

This lab manual explains:

- Fundamentals of Streaming Systems
- Apache Kafka Architecture (KRaft Mode)
- Apache Spark Structured Streaming
- Compatibility Matrix (Spark 4.1.1 + Kafka 4.1.1)
- Step-by-Step Installation Guide
- Integration Setup
- Classroom Architecture Flow

2. What is Streaming?

Traditional Processing (Batch)

- Data collected over time
- Processed later
- Example: End-of-day reports

Streaming Processing

- Continuous data flow
- Near real-time processing
- Example: Logs, IoT, Financial transactions

3. Why Do We Need Streaming?

Modern systems generate:

- Application logs
- Clickstream data
- Sensor data
- Financial transactions
- Social media feeds

Streaming allows: - Low-latency processing - Scalable ingestion - Fault-tolerant systems

4. Apache Kafka Fundamentals

Kafka is a distributed event streaming platform.

Core Concepts:

1. Broker

Kafka server that stores messages.

2. Topic

Logical channel where messages are published.

3. Partition

Topics are split into partitions for parallelism.

4. Producer

Application that sends messages to Kafka.

5. Consumer

Application that reads messages.

6. Consumer Group

Multiple consumers sharing workload.

5. Kafka Architecture (KRaft Mode)

Kafka 4.x uses KRaft (Kafka Raft Metadata Mode).

What Changed?

- No Zookeeper required
- Metadata managed internally
- Cleaner architecture
- Simplified deployment

Flow:

Producer → Kafka Broker → Topic (Partitioned) → Consumer

6. Apache Spark Structured Streaming

Spark Structured Streaming:

- Built on Spark SQL engine
- Micro-batch processing model
- Fault tolerant
- Exactly-once guarantees (with checkpointing)

Processing Model:

Input Stream → Transformations → Output Sink

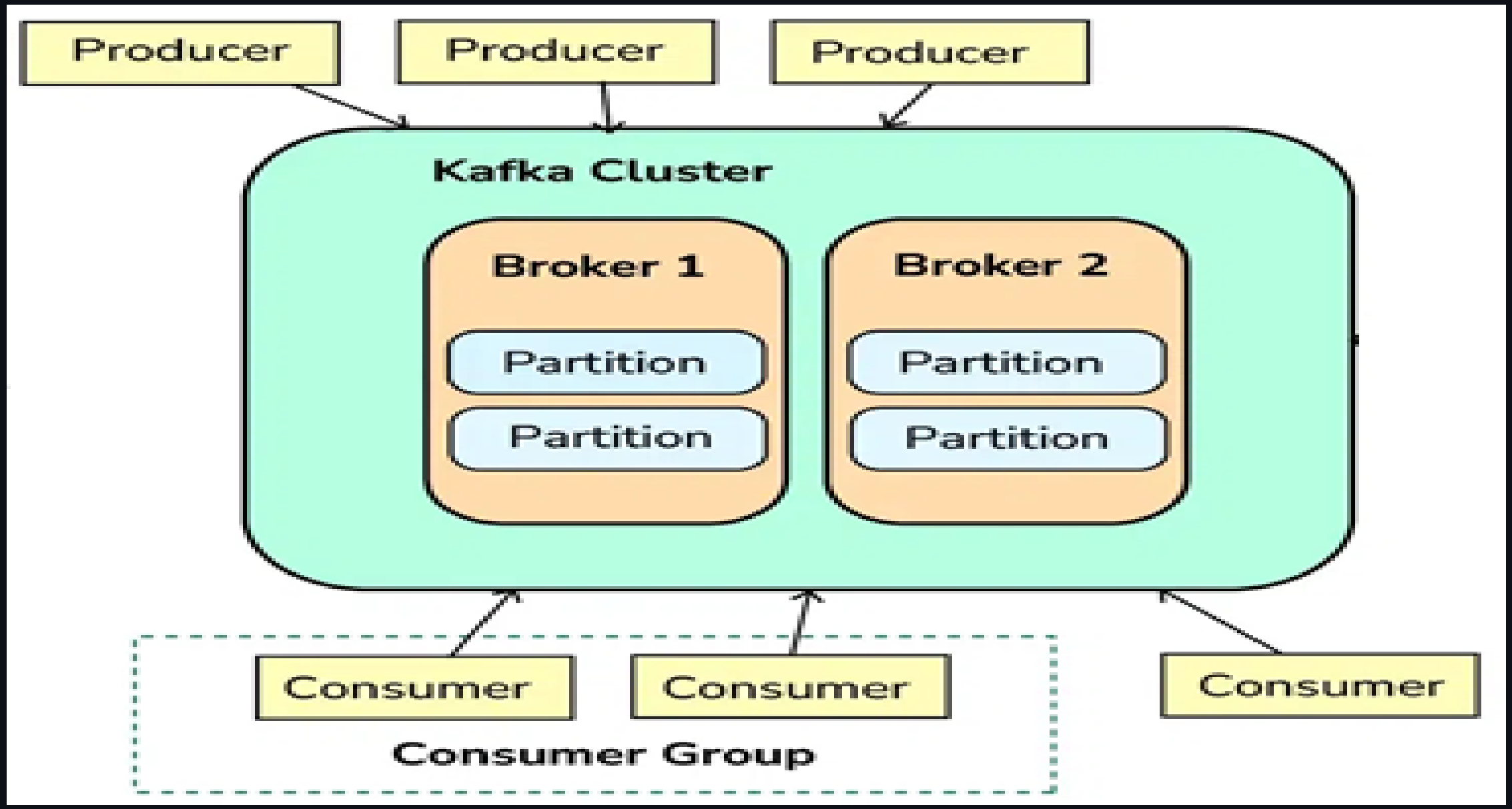
7. How Spark and Kafka Work Together

Architecture:

Producer → Kafka Topic → Spark Streaming → Sink

Internal Workflow:

1. Producer sends data to Kafka.
2. Kafka stores messages in partitions.
3. Spark connects via Kafka connector.
4. Spark reads data as stream DataFrame.
5. Transformations applied.
6. Results written to output sink.


Producer
Producer
Producer
Kafka Cluster
Broker 1
Partition
Partition
Broker 2
Partition
Partition
Consumer
Consumer
Consumer
Consumer Group

8. Compatibility Matrix

Environment:

- Java 17.0.12
- Spark 4.1.1 (Scala 2.13)
- Hadoop 3
- Kafka 4.1.1 (Scala 2.13 build)

Critical Rule:

Scala version of:

- Spark
- Kafka
- Spark-Kafka connector

MUST MATCH.

Correct Connector:

org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1

9. Installation Guide -- Kafka 4.1.1

Step 1: Download

Download: kafka_2.13-4.1.1.tgz

Step 2: Extract

Linux: tar -xzf kafka_2.13-4.1.1.tgz

Windows: Extract ZIP to C:`\kafka`

10. Setup KRaft Mode

a) Generate Cluster ID (Windows)

```
C:\kafka_2.13-4.1.1\bin>cd windows
```

```
C:\kafka_2.13-4.1.1\bin\windows>kafka-storage.bat random-uuid
2026-02-24T10:18:47.601916800Z main ERROR Reconfiguration failed: No configuration found for '266474c2' at 'null' in 'null'
guqbLBY9SkOY7nuDIhLnCA
```

Note the UUID just created (generated Cluster ID ✅).

- The UUID (cluster ID) is only required when you format the storage directories for the first time using `kafka-storage.bat format`

The ERROR line above it is:

A harmless Log4j logging configuration warning:

- Not a Kafka failure
- Can be safely ignored
- Kafka 4.x sometimes prints this logging warning in Windows.

b) Change Log Directory (Important on Windows)

Open :C:\kafka_2.13-4.1.1\config\server.properties

replace the line below

```
log.dirs=/tmp/kraft-combined-logs
```

with

```
log.dirs=C:/kafka_2.13-4.1.1/kraft-logs
```

c) Format Storage

❌

```
C:\kafka_2.13-4.1.1\bin\windows>
kafka-storage.bat format -t guqbLBY9SkOY7nuDIhLnCA -c ..\..\config\server.properties
```

```
2026-02-24T10:26:57.930594600Z main ERROR Reconfiguration failed:
No configuration found for '266474c2' at 'null' in 'null'
Because controller.quorum.voters is not set on this controller, you must specify one of the following:
 --standalone, --initial-controllers, or --no-initial-controllers.
```

✅

```
C:\kafka_2.13-4.1.1\bin\windows>
kafka-storage.bat format --standalone -t guqbLBY9SkOY7nuDIhLnCA -c ..\..\config\server.properties
```

Explanation of switches

format

- This subcommand initializes (formats) the Kafka log directories with a cluster ID.
- It’s required before starting a broker for the first time.

--standalone

- Runs the formatting process in standalone mode.
- This means the tool doesn’t try to connect to other brokers or a controller quorum; it just formats the local storage.
- Useful for single-node setups or when preparing each broker individually.

-t guqbLBY9SkOY7nuDlhLnCA

- The -t flag specifies the cluster ID (UUID) to embed into the storage metadata.
- All brokers in the same cluster must use the same cluster ID, so you typically generate it once with random-uuid and reuse it here.

-c ....\config\server.properties

- The -c flag points to the Kafka configuration file.
- This file contains broker settings like log directory paths, ports, and other runtime parameters.
- The formatter uses it to know which directories to initialize.

```
2026-02-24T10:54:58.897007600Z main ERROR Reconfiguration failed:
No configuration found for '266474c2' at 'null' in 'null'
Formatting dynamic metadata voter directory
C:/kafka_2.13-4.1.1/kraft-logs with metadata.version 4.1-IV1.
```

That means:

- 👉 Storage formatted successfully
- 👉 Metadata directory created
- 👉 Cluster ID registered
- 👉 Kafka ready to start

The "ERROR Reconfiguration failed" is just the same harmless Log4j warning you saw earlier. Ignore it.

11. Start Kafka Server

Linux: bin/kafka-server-start.sh config/kraft/server.properties

Windows:

```
C:\kafka_2.13-4.1.1\bin\windows>kafka-server-start.bat ^
..\..\config\server.properties
```

- caret symbol (^) as a line continuation character

```
DEPRECATED: A Log4j 1.x configuration file has been detected, which is no longer recommended.
... (kafka.network.ConnectionQuotas)
```

```
[2026-02-24 16:30:03,004] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

If no errors → Kafka running on localhost:9092 [Do NOT close this window.]

12. Create Topic

```
kafka-topics.bat --create --topic test-topic --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

If it says:

```
Created topic test-topic.
```

Kafka is fully working 🎉

Verify:

```
kafka-topics.bat --list --bootstrap-server localhost:9092
```

If you see *test-topic* You are production-ready.

```
kafka-topics.bat --create --topic test-topic --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

- This command uses the topic management tool of Apache Kafka.

1️⃣ kafka-topics.bat

This is the Windows script used to: - Create topics, Delete topics, List topics, Describe topics.

2️⃣ --create
👉 Action switch. It tells Kafka: "Create a new topic."
Other possible actions: `--list, --describe, --delete, --alter`

3️⃣ --topic
This specifies the name of the topic (A logical channel or stream where messages are stored.) [Eg. orders, payments, iot-data, logs].

4️⃣ --bootstrap-server localhost:9092

This tells the tool: Which Kafka broker should I connect to?

Breakdown: localhost → Your machine ; 9092 → Broker port (PLAINTEXT listener)

Important:

- This does NOT mean the topic is created only on that server.
- It just means: "Start communication with this broker."
- Kafka will handle the rest internally.

5️⃣ --partitions 1

Defines how many partitions the topic will have.

- Partition: A topic is divided into partitions.
- Each partition: Is an ordered log
- Stores messages sequentially: Enables parallel processing

If you set: `--partitions 3` then, Topic will have 3 logs

- Consumers can read in parallel
- Throughput increases
- For learning, 1 is fine.

6️⃣ --replication-factor 1

This defines, How many copies of each partition exist.

If you have:

- 1 broker → must use replication factor 1
- 3 brokers → you can use replication factor 2 or 3

Example: `--replication-factor 3`

Means: Each partition has 3 copies on different brokers.

- But since you are running single-node Kafka, replication must be 1.

🎯 Why These Switches Matter

Switch	Purpose
--create	Create new topic
--topic	Name of topic
--bootstrap-server	Which broker to connect to
--partitions	Parallelism level
--replication-factor	Fault tolerance level

- Kafka Process
  - Broker (PLAINTEXT://localhost:9092)
  - Controller (CONTROLLER://localhost:9093)
  - Metadata stored in kraft-logs folder

```
                  ┌──────────────────────────────┐
                  │          Kafka Node          │
                  │   (process.roles=both)       │
                  │                              │
Producer ────►    │  Broker (9092)               │
Consumer ◄────    │                              │
                  │  Controller (9093)           │
                  │  Metadata (KRaft logs)       │
                  └──────────────────────────────┘
```

🏗️ Overall Architecture (Single Node Setup)

We configured: `process.roles=broker,controller`

- 👉 The SAME process is acting as: Broker & Controller
- But internally they are logically separate components.

1️⃣ Broker: PLAINTEXT://localhost:9092 ; This is the data layer.

- What Broker Does:
  Stores messages, Handles producers, Handles consumers, Maintains partitions, Manages replication

When you run: `kafka-topics.bat --bootstrap-server localhost:9092` You are talking to the Broker.

🔄 Data Flow Example: `Producer → Broker (9092) → Partition log file → Consumer`

Physically stored inside: `C:/kafka_2.13-4.1.1/ My-kraft-logs/`
Each partition becomes a directory.

2️⃣ Controller: CONTROLLER://localhost:9093

- This is the brain of the cluster.

The controller: Creates topics, Assigns partitions to brokers, Elects leaders, Manages metadata, Handles failover

⚠️ Clients NEVER connect to controller directly.

It is used internally by Kafka nodes.

3️⃣ Metadata Store (KRaft)

Before Kafka 3.x: Kafka used Apache ZooKeeper.

Now in Kafka 4.x: ZooKeeper is removed. Kafka uses: 👉 KRaft (Kafka + Raft consensus)

- Where Metadata Is Stored?

Inside your log directory: `My-kraft-logs/`

There are: Metadata log, Topic metadata, Cluster ID, Partition leader info, ISR (in-sync replicas)

🧠 What Is Metadata?

Metadata includes: Topic names, Partition count, Replication factor, Leader broker, Offsets, ACLs, Configurations

- This is NOT your message data.
- This is control information.

🔥 What Happens When You Create Topic?

Step-by-step internally:

1. CLI connects to Broker (9092)
2. Broker forwards request to Controller
3. Controller:
   -- Updates metadata log
   -- Assigns partitions
   -- Writes metadata to KRaft log
4. Broker creates partition directories
5. Topic is ready

🔥 Why Two Ports?

Port	Role	Who Uses It
9092	Broker	Producers & Consumers
9093	Controller	Internal Kafka communication

🎯 Broker: Stores and serves data.

🎯 Controller: Manages cluster brain and metadata.

🎯 Metadata Store: Internal Raft log storing cluster state.

13. Test Producer & Consumer

- verify Kafka messaging works perfectly.

This isolates problems.

- If Producer/Consumer works → Kafka is healthy.
- If Spark fails later → issue is connector/config, not Kafka.

Do NOT close the window where Kafka server is running.

✅ STEP 1 — Open NEW CMD Window

```
cd C:\kafka_2.13-4.1.1\bin\windows>
```

✅ STEP 2 — Check Topic Exists

```
kafka-topics.bat --list --bootstrap-server localhost:9092
```

✅ STEP 3 — Start Consumer FIRST (Recommended Order)[Leave this window open.]

```
kafka-console-consumer.bat --topic test-topic --from-beginning --bootstrap-server localhost:9092
```

✅ STEP 4 — Start Producer (New CMD Window)

```
kafka-console-producer.bat --topic test-topic --bootstrap-server localhost:9092
```

Now type:

```
Hello Kafka
Testing...
```

✅ STEP 5 — Observe Consumer Window

```
Hello Kafka
Testing...
```

If you should immediately see:

🎉 Producer → Broker → Topic → Consumer working perfectly.

🧠 What Just Happened

```
Producer
   ↓
Broker (9092)
   ↓
Partition (test-topic-0)
   ↓
Consumer
```

Internally:

- Producer sends record to partition
- Kafka appends to log file
- Consumer reads offset sequentially

🔥 Concepts Just Demonstrated

You can now explain:

1️⃣ Topic: Logical message channel.

2️⃣ Partition: Physical append-only log.

3️⃣ Offset: Message position inside partition.

4️⃣ Pull Model: Consumer pulls data from broker.

🔥 Problem you might have

```
C:\kafka_2.13-4.1.1\bin\windows>
kafka-server-start.bat ..\..\config\server.properties
```

If above command output nothing, then try below:

```
C:\kafka_2.13-4.1.1\bin\windows>
kafka-run-class.bat kafka.Kafka ..\..\config\server.properties
```

If you got something like "Exceptionally slow controller event writeNoOpRecord"
It happens when:

- Old KRaft metadata exists
- Log directory was not properly cleaned
- Format was done with conflicting quorum configs earlier

🔥 Problem: Fix With a FULL CLEAN RESET

✅ STEP 1 — Stop Everything: (Close all Kafka windows.)

✅ STEP 2 — DELETE ALL Kafka Metadata (VERY IMPORTANT)

Delete completely: (Make sure the folder is completely gone.)

- C:\kafka_2.13-4.1.1\My-kraft-logs

✅ STEP 3 — Verify server.properties (Final Correct Version)

✅ STEP 4 — Generate New UUID `kafka-storage.bat random-uuid`

✅ STEP 5 — Format Fresh `kafka-storage.bat format --standalone -t YOUR_UUID -c ..\..\config\server.properties`

✅ STEP 6 — Start Kafka `kafka-server-start.bat ..\..\config\server.properties`

✅ Let’s Verify If Kafka Is Running

Open a NEW CMD window (don’t close the first one).

Run:

```
netstat -ano | findstr 9092
```

If you see something like:

```
TCP    0.0.0.0:9092     LISTENING
```

✅ Kafka is running correctly.

Integrate with Spark Structured Streaming:

✅ Step 1 — Download Spark (If Not Installed) verify the version information

```
C:\spark-4.1.1-bin-hadoop3\bin>spark-shell --version
```

✅ Step 2 — Add Kafka Package to Spark & Run Spark Streaming Job

```
spark-shell --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1
```

Notice:

- _2.13 ← because your Spark uses Scala 2.13
- 4.1.1 ← must match Spark version exactly

⚠️ Important Notes for Spark 4.x

Spark 4.x is newer, so:

- Make sure you have internet access
- Firewall should not block Maven download
- Scala version MUST match (ours is 2.13)

Version Matching Is Critical: Mismatch = ClassNotFound errors.

✅ Step 3 — Verify It Loaded Correctly Inside Spark shell: ```spark.read.format("kafka") ```
If no error → connector loaded.

🔍 What Happenecd

First time:

- Spark downloads jars from Maven, - You will see many dependency logs
- It may take 1–2 minutes, - After that, it caches locally.

🚀 STEP 4 — Read Stream From Kafka

```
import org.apache.spark.sql.functions._
val kafkaDF = spark
  .readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("subscribe", "test-topic")
  .option("startingOffsets", "latest")
  .load()
```

🧠 What This Does

Spark now acts like a Kafka consumer group.

But data is still in binary format.

Schema looks like:

```
key: binary
value: binary
topic: string
partition: int
offset: long
timestamp: timestamp
```

🚀 STEP 5 — Convert Binary to String

```
val readableDF = kafkaDF.selectExpr(
  "CAST(key AS STRING)",
  "CAST(value AS STRING)",
  "topic",
  "partition",
  "offset",
  "timestamp"
)
```

🎯 TEST IT Now:

1️⃣ Keep Spark running -> 2️⃣ Open new CMD -> 3️⃣ Run producer:

```
kafka-console-producer.bat --topic test-topic --bootstrap-server localhost:9092
```

Type: something, You should immediately see output in Spark console 🎉

🏗️ What Just Happened Internally

Architecture now / kafka pipeline:

```
Producer → Kafka (9092) → Spark Streaming Engine → Console Sink
```

Spark:

- Creates its own consumer group
- Tracks offsets
- Processes micro-batches

🔥 Add Checkpointing (Production Pattern)

Without checkpointing → offsets lost on restart.

Stop current query `Ctrl + C`.

Then run:

```
val query = readableDF.writeStream
  .format("console")
  .outputMode("append")
  .option("checkpointLocation", "C:/spark-checkpoint")
  .start()
query.awaitTermination()
```

Now Spark stores: Offsets >> State >>Progress

- This is real production ingestion pattern.

🧠 Ingestion Patterns You Just Implemented

Pattern 1 — Direct Stream Processing

```
Kafka → Spark → Console
```

Pattern 2 — Fault Tolerant Ingestion

```
Kafka → Spark (Checkpoint) → Sink
```

Data ingestion means:

The process of collecting and importing data into a system for processing or storage.

Final Architecture:

Producer -> Kafka Topic (Partitioned) -> Spark Structured Streaming -> Console / File / Database

End of unit 5